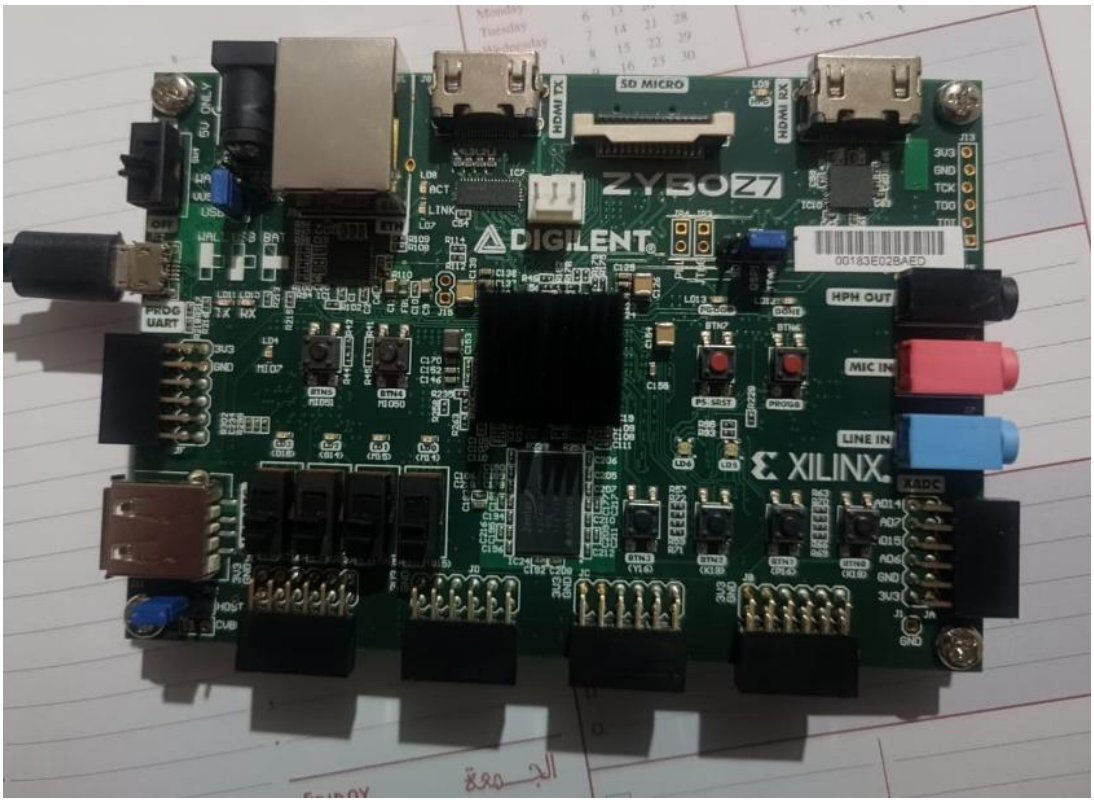
FPGA

Student Names
Abdelrhman Khaled Fouad
Ahmed Haitham Othman
Ahmed Hossam Rafik
Mohamed Adel Abdelrahem
Mohamed Ahmed Mohamed Hussien
Moustafa Saad Dawood
Youssef Mohamed Mamdouh

content	page
Our FPGA	2
Logic analyzer	3-4
How to generate inputs for FPGA	5-6
Real test (AND GATE)	7-8
ILA test	9-10

OUR FPGA

Part used in vivado is "xc7z020-clg400-2"



Logic analyzer

A **Logic Analyzer** is a diagnostic tool used to capture, visualize, and analyze digital signals. Unlike an oscilloscope—which measures analog voltage over time to show signal integrity (like noise or ringing)—a logic analyzer is focused on **timing and logic states** (0s and 1s) across many channels simultaneously.

In the context of FPGAs, logic analyzers come in two forms: external hardware and Internal Logic Analyzers (ILAs).

---

1. How a Logic Analyzer Works (The Basics)

A logic analyzer functions by sampling digital inputs at a specific clock rate and comparing the voltage to a threshold.

- **Sampling:** It looks at the signal at every "tick" of a sample clock.
- **Thresholding:** If the voltage is above a certain level, it's recorded as a 1; below that, it's a 0.
- **Triggering:** You define a specific condition (e.g., "start recording when the reset signal goes low and data_valid goes high").
- **Memory:** The captured bits are stored in memory and then displayed as a waveform.

---

2. The Internal Logic Analyzer (ILA) in FPGAs

Most modern FPGA development (using tools like Vivado or Quartus) utilizes an **Embedded/Integrated Logic Analyzer (ILA)**. This is a piece of intellectual property (IP) core that you "instantiate" directly into your RTL code.

How it operates inside the chip:

1. **Probing:** You "attach" the ILA to the internal wires (nets) you want to monitor in your SystemVerilog or VHDL design.

2. **Resource Usage:** The ILA uses the FPGA's internal **Block RAM (BRAM)** to store the captured signal data and utilizes some of the FPGA's logic cells to manage triggering.
3. **Clocking:** The ILA runs on a clock you provide (usually the design's main system clock). It captures the state of the probes on every rising edge.
4. **Communication:** Once the trigger condition is met and the BRAM is full, the data is sent over the **JTAG cable** to your computer.
5. **Software Visualization:** The IDE (Vivado/Quartus) decodes that data and shows you a waveform of what happened *inside* the chip in real-time.

---

3. Comparison External vs internal

Feature	External Logic Analyzer	Internal Logic Analyzer (ILA)
Visibility	Only signals connected to physical pins.	Any internal wire/register in the code.
Cost	Can be expensive (hundreds to thousands of dollars).	Free (included with FPGA vendor tools).
Signal Integrity	Can affect the signal due to probe capacitance.	No physical impact on the signal.
Memory	Massive external storage.	Limited by the FPGA's internal BRAM.
Setup	Requires physical "flying leads" or Mictor cables.	Configured via software and JTAG.

How to generate inputs for FPGA design

1. Memory Initialization (Static Method)

Definition: Memory Initialization is a technique where input data is pre-calculated in a software environment (like MATLAB) and stored directly into the FPGA's internal **Block RAM (BRAM)** or **Read-Only Memory (ROM)** during the bitstream generation process. The FPGA design reads this "frozen" data at runtime as if it were a real-time input stream.

- **Mechanism:** Data is formatted into a Coefficient (**COE**) file or Memory Initialization File (**MIF**). This file is linked to a RAM/ROM IP core during the synthesis stage in Vivado.
- **Workflow:** MATLAB → .COE file → Vivado Synthesis → FPGA Execution.

2. FPGA-in-the-Loop (Dynamic Method)

Definition: FPGA-in-the-Loop (FIL) is an integrated co-simulation strategy that connects a high-level mathematical environment (MATLAB/Simulink) with physical hardware. It allows the software to act as a testbench that streams data to the FPGA, waits for the hardware to process it, and retrieves the results for immediate analysis.

- **Mechanism:** A communication wrapper (via JTAG or Ethernet) is automatically inserted into the FPGA design. MATLAB manages the synchronization between the software "source" and the hardware "sink."
- **Workflow:** MATLAB ←→ JTAG/Ethernet ←→ Physical Zybo Board.

3. Manual Hardware Interfacing (Physical Method)

Definition: Manual Hardware Interfacing is the most direct form of FPGA interaction, where internal digital signals are mapped to physical components on the development board, such as slide switches, push buttons, and LEDs. This method allows the user to manually control logic states in real-time by physically toggling inputs.

- **Mechanism:** User constraints are defined in a Xilinx Design Constraints (**XDC**) file, which maps the RTL input ports to the specific package pins of the FPGA (e.g., the 4 switches on the Zybo Z7-20). Debouncing logic is often required to ensure a clean transition from mechanical switches.
- **Workflow:** Hardware Switches → FPGA Package Pins → RTL Logic → LED/Display Output.

Why this method is not used in our system:

- **Data Volume:** A 4096-point FFT requires thousands of complex data points per frame. Manually toggling switches to input even a single frame would be physically impossible.
- **Clock Speed:** Physical switches operate at human speeds (Hz), whereas our ISAC signal processing pipeline operates at high frequencies (e.g.,100 MHz).
- **Precision:** Switches can only provide simple binary inputs, making them unsuitable for representing the high-precision fixed-point complex numbers required for OFDM and Radar signal analysis.

Comparison Table for Hardware Verification

Feature	**Memory Initialization (COE)**	**FPGA-in-the-Loop (FIL)**
Data Nature	**Static:** Fixed dataset embedded in bitstream.	**Dynamic:** Real-time data streaming.
Flexibility	**Low:** Must re-synthesize/re-program to change data.	**High:** Change inputs in MATLAB instantly.
Communication	None (Internal to chip).	Active (JTAG, Ethernet, or USB).
Resource Usage	Consumes significant **BRAM** for large datasets.	Consumes logic for the **Communication Wrapper**.
Processing Speed	**Full Speed:** Runs at the actual FPGA clock.	**Limited:** Throttled by the cable interface speed.
Complexity	Very Simple (HDL only).	Medium (Requires MATLAB HDL Verifier).
Best Use Case	Unit testing a single 4096-pt FFT frame.	Testing ISAC algorithms with varying noise/targets.

Our simple test (AND GATE)

Design

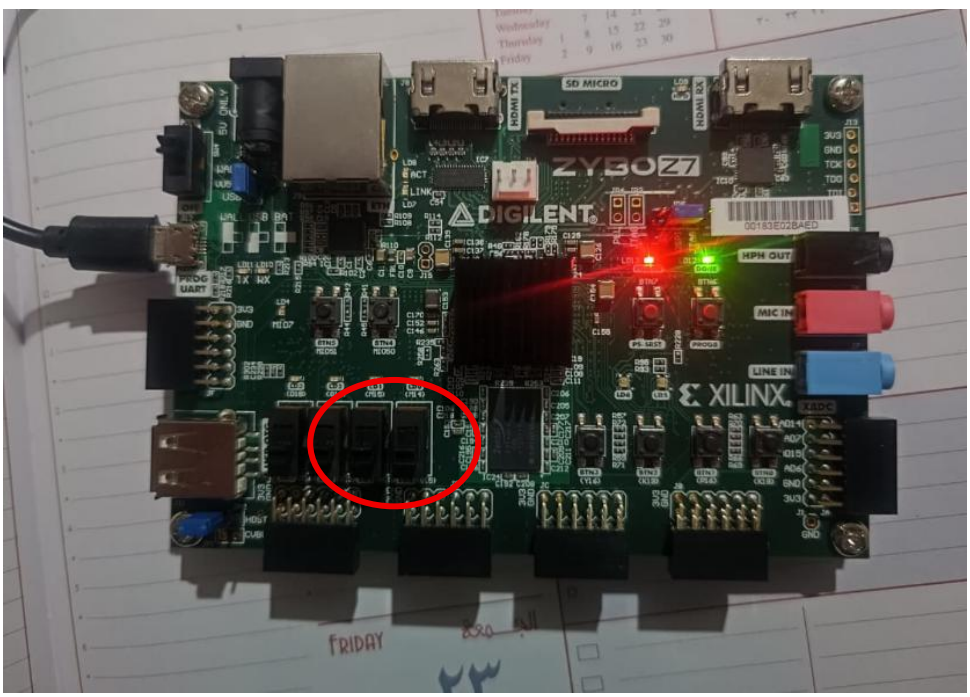
```
module and_gate (input i_a,i_b , output o_c);
and(o_c,i_a,i_b);
endmodule
```

Constraint file

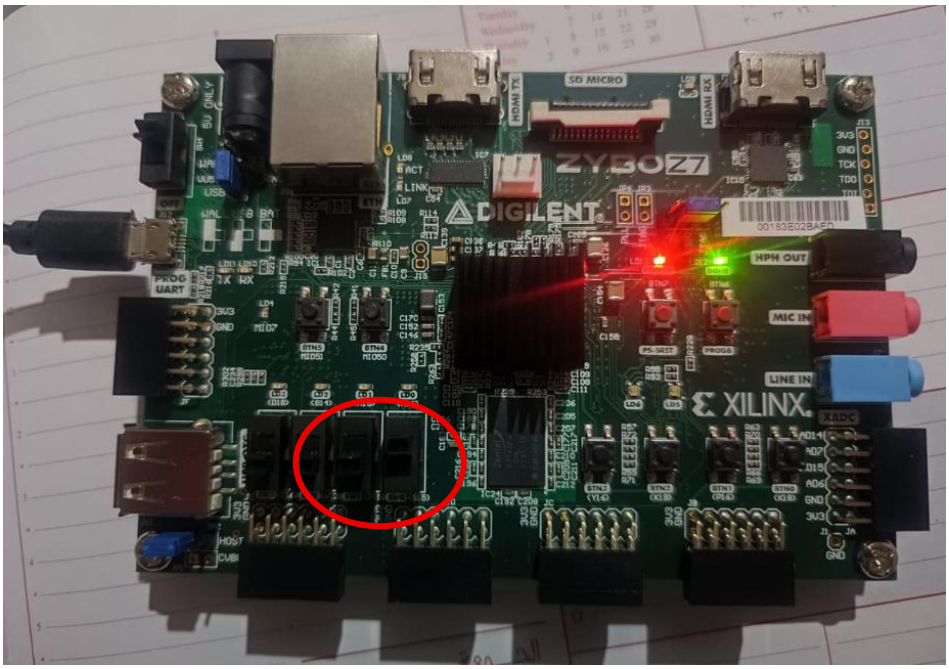
```
set_property -dict { PACKAGE_PIN G15   IOSTANDARD LVCMOS33 } [get_ports { i_a }]; #IO_L19N_T3_VREF_35 Sch=sw[0]
set_property -dict { PACKAGE_PIN P15   IOSTANDARD LVCMOS33 } [get_ports { i_b }]; #IO_L24P_T3_34 Sch=sw[1]
set_property -dict { PACKAGE_PIN M14   IOSTANDARD LVCMOS33 } [get_ports { o_c }]; #IO_L23P_T3_35 Sch=led[0]
```

Real FPGA test

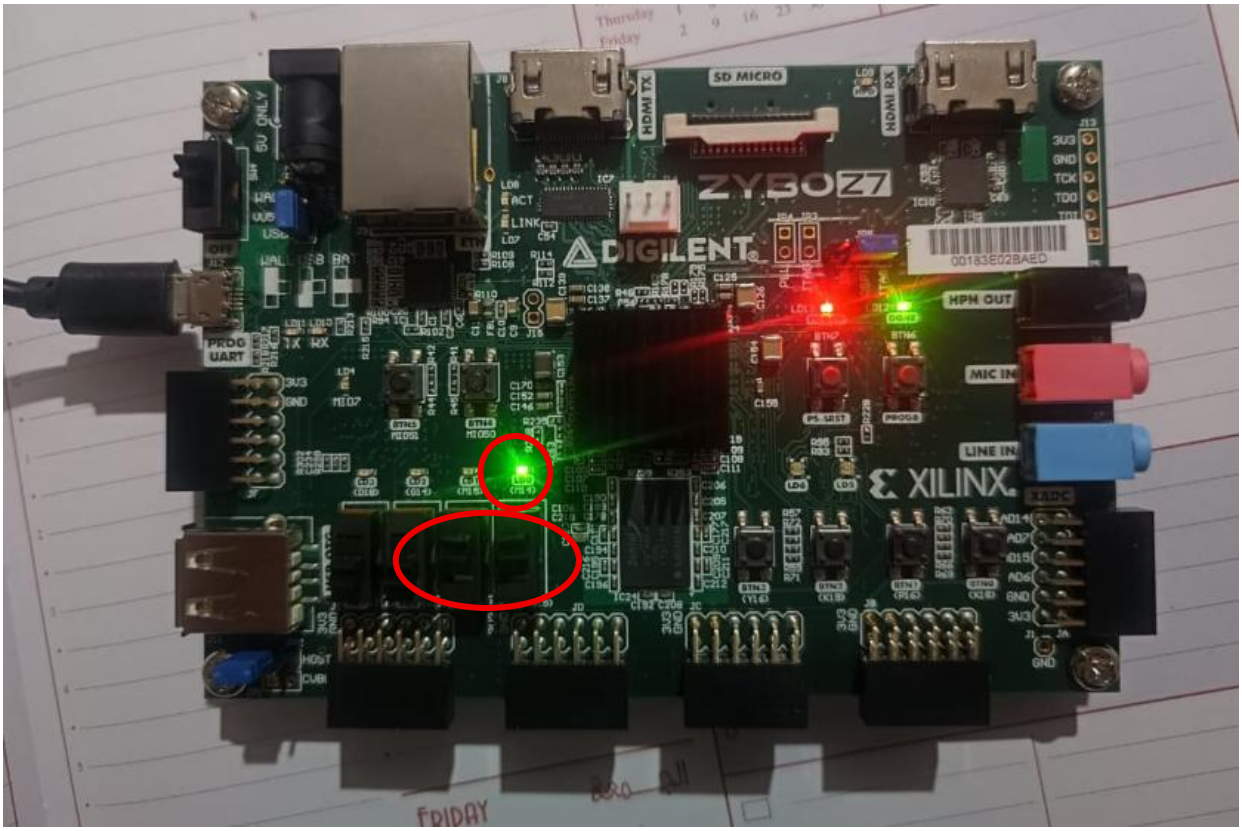
First FPGA with all switch low



Second one switch low and one high



Third both switches are high so the LED is now high



ILA test

Design

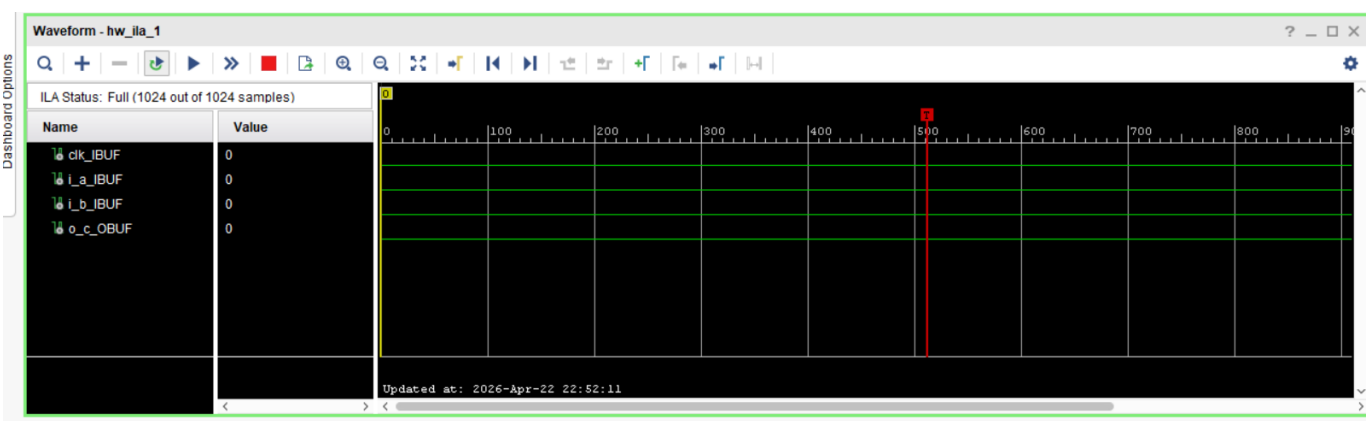
```
module and_gate (input i_a,i_b,clk, output reg  o_c );

always @ (posedge clk) begin

o_c <= i_a & i_b;

end

endmodule
```

Constraint file

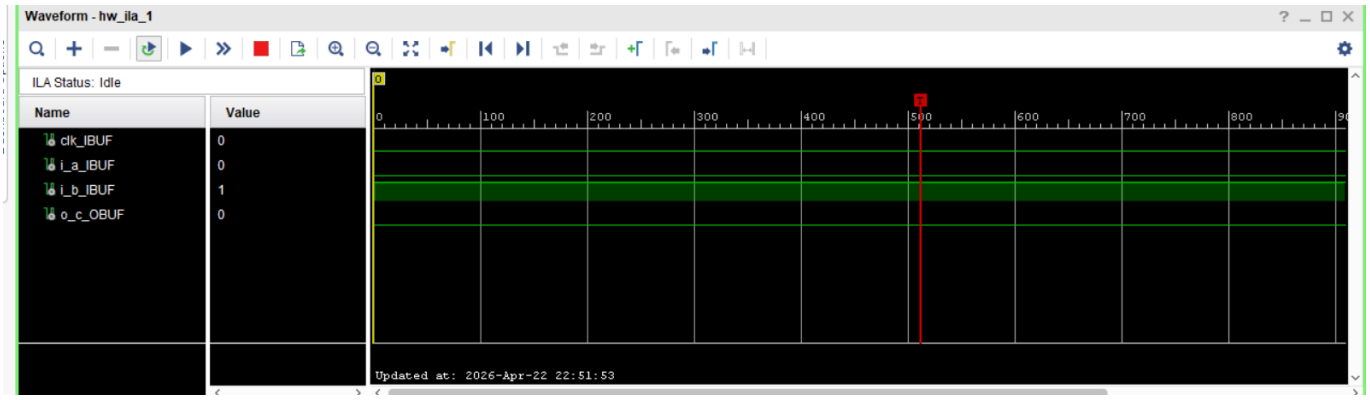
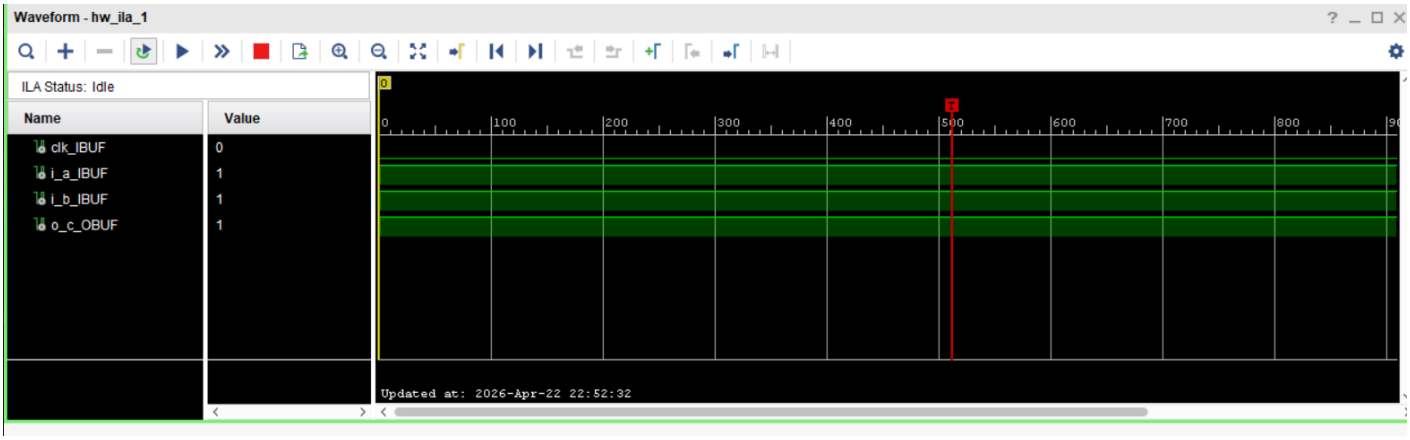
```
set_property -dict { PACKAGE_PIN G15   IOSTANDARD LVCMOS33 } [get_ports { i_a }];
#IO_L19N_T3_VREF_35 Sch=sw[0]

set_property -dict { PACKAGE_PIN P15   IOSTANDARD LVCMOS33 } [get_ports { i_b }];
#IO_L24P_T3_34 Sch=sw[1]

set_property -dict { PACKAGE_PIN M14   IOSTANDARD LVCMOS33 } [get_ports { o_c }];
#IO_L23P_T3_35 Sch=led[0]

set_property -dict { PACKAGE_PIN K17   IOSTANDARD LVCMOS33 } [get_ports { clk }];
#IO_L12P_T1_MRCC_35 Sch=sysclk

create_clock -add -name sys_clk_pin -period 8.00 -waveform {0 4} [get_ports { clk }];
```

ILA waveform snippets

Both inputs are zero



one switch is high



Both switches are high and output is also high